# INSTITUTO NEBRIJA DE FORMACIÓN PROFESIONAL

**MEMORIA TÉCNICA • PRÁCTICA SUPABASE & IONIC**

| CICLO FORMATIVO | MÓDULO PROFESIONAL | ALUMNO | STACK TECNOLÓGICO |
|---|---|---|---|
| 2º DAM (Aplicaciones Multiplataforma) | Desarrollo de Interfaces (Código 0488) | Luis Lázaro Pimentel | Ionic 8 • Angular 18 • Supabase |

GH **REPOSITORIO OFICIAL EN GITHUB (EDITABLE)**
`https://github.com/Thepimen/dam-crm-app`

▲ **DESPLIEGUE EN PRODUCCIÓN VERCEL (EDITABLE)**
`https://dam-crm-app.vercel.app`

## 1. ARQUITECTURA DE SEGURIDAD RBAC, RLS, LOGIN BAJO DEMANDA Y STORAGE BUCKET

`dam-crm-app`

### 1. RBAC y Trigger Automático en PostgreSQL

Definición del tipo `public.user_role ('admin', 'user')` y tabla `public.profiles` vinculada a `auth.users`. El trigger `handle_new_user()` asigna automáticamente el rol `'admin'` al registrar cualquier usuario, garantizando pruebas fluidas sin bloqueos.

### 2. Políticas RLS Diferenciadas (Lectura vs Escritura)

La tabla `public.clients` permite lectura inicial abierta ( `FOR SELECT TO anon, authenticated USING (true)` ), mientras que las operaciones CUD exigen subconsulta verificada de rol `'admin'` en `public.profiles`.

### 3. Interceptor de Login bajo Demanda en CUD

Si el usuario anónimo pulsa *Guardar/Insertar*, *Editar* o *Eliminar*, `HomePage` retiene la operación en `pendingAction`, despliega un `ion-modal` contra Supabase Auth, sincroniza las Signals y ejecuta automáticamente la acción pendiente.

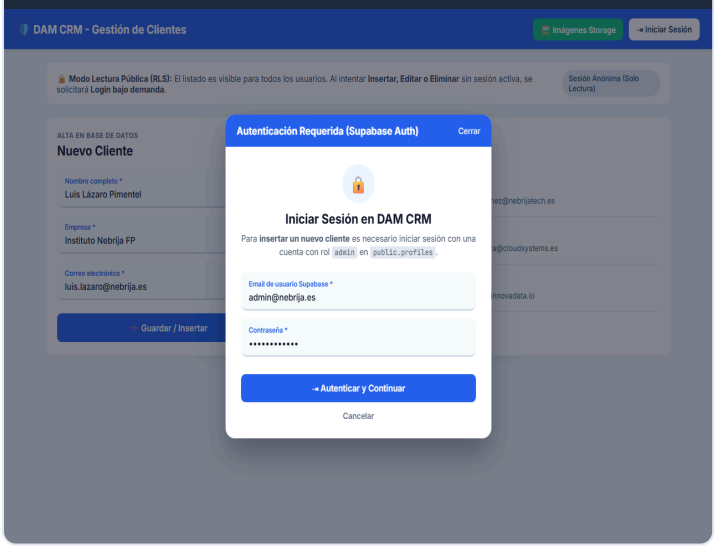
### 4. Galería Conectada a Supabase Storage + SPA Vercel

La ruta `/galeria` lista objetos del bucket público `'gallery'` con `list()` + `getPublicUrl()` en un `ion-grid` responsivo de `ion-card`, permitiendo subida autenticada y despliegue SPA mediante rewrites en `vercel.json`.

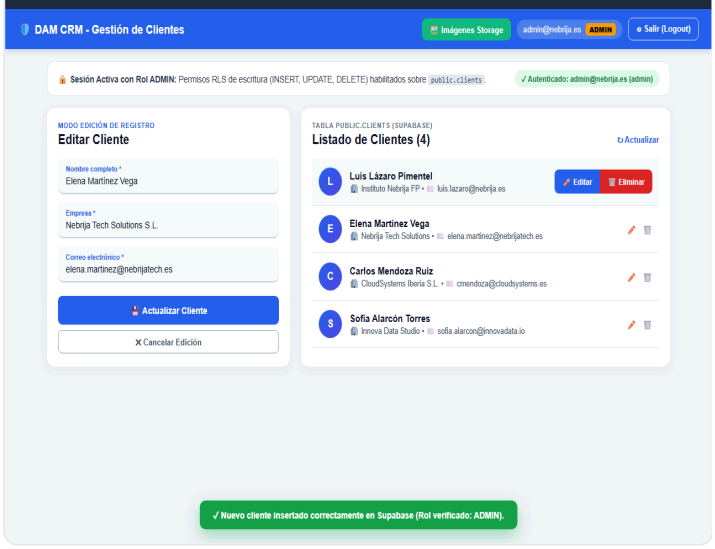
## 2. EVIDENCIAS DE FUNCIONAMIENTO DE LA APLICACIÓN Y BACKEND EN SUPABASE

`docs/captura1.png – docs/captura4.png`

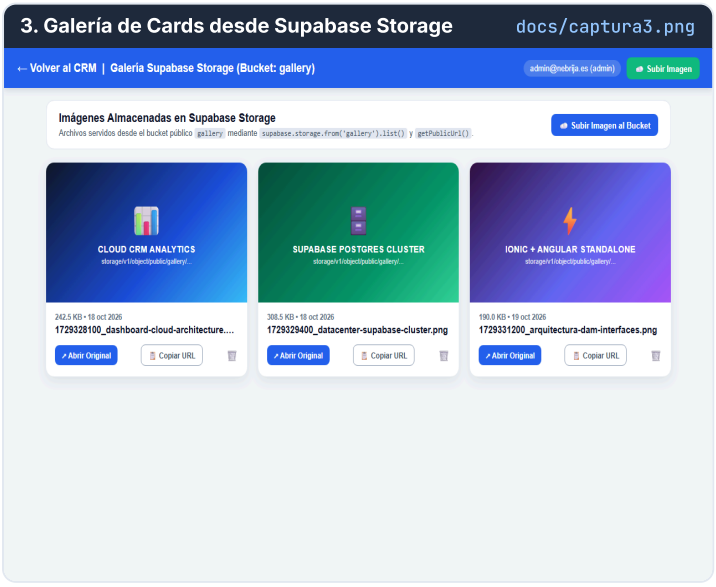
### 1. Modal de Login al intentar CUD — `docs/captura1.png`



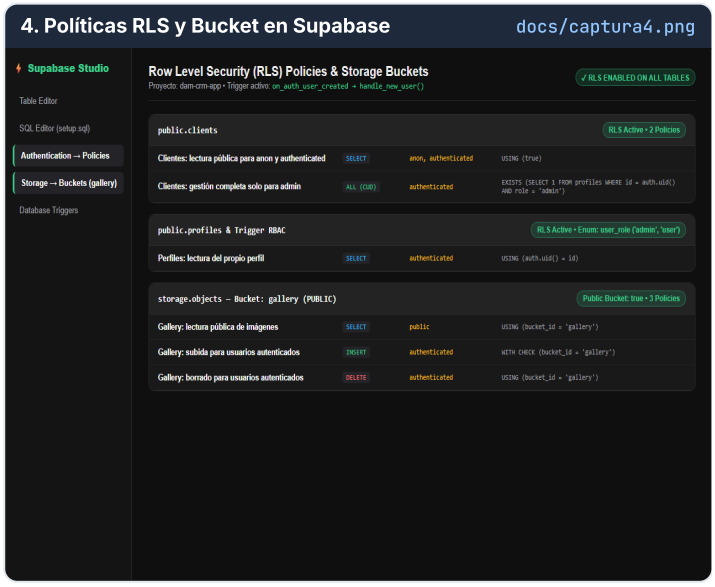

### 2. CRUD de Clientes autenticado como Admin — `docs/captura2.png`




### 3. Galería de Cards desde Supabase Storage — `docs/captura3.png`




### 4. Políticas RLS y Bucket en Supabase — `docs/captura4.png`



## 3. IMPLEMENTACIÓN TÉCNICA — FRAGMENTOS CLAVE DEL CÓDIGO FUENTE

SQL RLS • Angular Signals • Login Interceptor • Storage Cards

### 3.1. Políticas RLS de Lectura Pública, Escritura RBAC Admin y Storage Bucket

supabase/setup.sql

```sql
-- Lectura abierta en public.clients para visualizar listado antes de iniciar sesión
CREATE POLICY "Clientes: lectura pública para anon y authenticated"
  ON public.clients FOR SELECT TO anon, authenticated USING (true);

-- Escritura (INSERT, UPDATE, DELETE) restringida a usuarios autenticados con rol 'admin'
CREATE POLICY "Clientes: gestión completa solo para admin"
  ON public.clients FOR ALL TO authenticated
  USING (EXISTS (SELECT 1 FROM public.profiles WHERE id = auth.uid() AND role = 'admin'))
  WITH CHECK (EXISTS (SELECT 1 FROM public.profiles WHERE id = auth.uid() AND role = 'admin'));

-- Bucket público 'gallery' y políticas RLS sobre storage.objects
INSERT INTO storage.buckets (id, name, public) VALUES ('gallery', 'gallery', true) ON CONFLICT (id) DO UPDATE SET public = true;
CREATE POLICY "Gallery: lectura pública de imágenes" ON storage.objects FOR SELECT TO public USING (bucket_id = 'gallery');
CREATE POLICY "Gallery: subida para usuarios autenticados" ON storage.objects FOR INSERT TO authenticated WITH CHECK (bucket_id = 'gallery');
```

### 3.2. Servicio Único Reactivo con Signals, Auth en Tiempo Real y Storage Bucket

src/app/services/supabase.service.ts

```ts
export class SupabaseService {
  readonly currentUser = signal<User | null>(null);
  readonly userRole = signal<'admin' | 'user' | null>(null);

  async login(email: string, password: string): Promise<User> {
    const { data, error } = await this.supabase.auth.signInWithPassword({ email: email.trim(), password });
    if (error || !data.user) throw error || new Error('Error de autenticación');
    this.currentUser.set(data.user);
    await this.loadRole(data.user.id);
    return data.user;
  }

  async getBucketImages(): Promise<StorageImage[]> {
    const { data, error } = await this.supabase.storage.from('gallery').list('', { limit: 100 });
    if (error) throw error;
    return (data ?? [])
      .filter(file => file.name !== '.emptyFolderPlaceholder' && !file.name.startsWith('.'))
      .map(file => ({
        name: file.name,
        url: this.supabase.storage.from('gallery').getPublicUrl(file.name).data.publicUrl,
        created_at: file.created_at,
        size: file.metadata?.['size']
      }));
  }
}
```

### 3.3. Interceptor de Seguridad (Login bajo Demanda al Insertar, Editar o Eliminar)

src/app/home/home.page.ts

```ts
async onSubmitClient(): Promise<void> {
  if (this.clientForm.invalid) { this.clientForm.markAllAsTouched(); return; }
  if (!this.currentUser()) {
    this.openLoginForPendingAction({ type: 'save' }, 'insertar o actualizar cliente');
    return;
  }
  await this.executeSaveClient();
}

async onConfirmLogin(): Promise<void> {
  const { email, password } = this.loginForm.value;
  const user = await this.supabaseService.login(email, password);
  this.isLoginModalOpen.set(false);
  await this.presentToast(`Sesión iniciada como ${user.email} (Rol: ${this.userRole()})`, 'success');

  if (this.pendingAction && this.userRole() === 'admin') {
    if (this.pendingAction.type === 'save') await this.executeSaveClient();
    if (this.pendingAction.type === 'edit') this.populateFormForEdit(this.pendingAction.client);
    if (this.pendingAction.type === 'delete') await this.executeDeleteClient(this.pendingAction.client);
    this.pendingAction = null;
  }
}
```

### 3.4. Maquetación Responsiva de Cards con Control Flow Moderno (@for / @empty)

src/app/gallery/gallery.page.html

```html
<ion-grid class="ion-no-padding">
  <ion-row>
    @for (img of images(); track img.name) {
      <ion-col size="12" size-sm="6" size-md="4">
        <ion-card class="gallery-card">
          <div class="image-wrapper"><img [src]="img.url" [alt]="img.name" /></div>
          <ion-card-header>
            <ion-card-subtitle>{{ formatSubtitle(img) }}</ion-card-subtitle>
            <ion-card-title>{{ img.name }}</ion-card-title>
          </ion-card-header>
          <ion-card-content>
            <ion-button size="small" (click)="openOriginalImage(img.url)">Abrir Original</ion-button>
            <ion-button size="small" fill="outline" (click)="copyPublicUrl(img.url)">Copiar URL</ion-button>
          </ion-card-content>
        </ion-card>
      </ion-col>
    } @empty {
      <ion-col size="12"><div class="empty-gallery">El bucket gallery aún no contiene imágenes.</div></ion-col>
    }
  </ion-row>
</ion-grid>
```